Contents

Diffusion Learnings

Table of contents

Chapter 1: Image codecs — VAE → VQ-VAE → VQGAN → MoVQ (and where latent diffusion sits)

1.1 The big picture: two separate problems

1.2 The evolution of the image codec

1.3 Architecturally, these are mostly CNN/ResNet autoencoders

1.4 What makes a VAE a VAE?

1.5 The VAE problem: KL can become too successful (posterior collapse)

1.6 VQ-VAE's radically different idea: make z discrete

1.7 The VQ-VAE training problem: nearest neighbour kills the gradient

1.8 Straight-through estimator: lie in backward, not forward

1.9 VQ-VAE has three learning jobs

1.10 Why codebook + commitment separately?

1.11 EMA gives an even cleaner interpretation of codebook learning

1.12 Why pixel MSE alone can produce blurry reconstruction

1.13 Three complementary reconstruction pressures

1.14 LPIPS from beginning to end

1.15 PatchGAN is a CNN critic of local realism

1.16 VQGAN = VQ representation + perceptual realism

1.17 MoVQ: what was left to improve?

1.18 MoVQ also injects the codes repeatedly into the decoder

1.19 The important latent distinction (three different "latents")

1.20 This explains the evolution of the actual generation systems

1.21 Why the DALL·E 2 prior can help diversity

1.22 The final mental map

Chapter 2: Training mechanics — normalization, CNNs, and the Adam/AdamW family

2.1 CNN translation behavior: equivariance vs invariance

2.2 Normalization: BatchNorm / GroupNorm / LayerNorm / RMSNorm

2.3 Adam: an adaptive per-coordinate learning rate

2.4 Why large normalized weights can still be undesirable

2.5 L2 regularization

2.6 Why AdamW exists

2.7 Weight decay does not normally flip signs

Chapter 3: Diffusion sampling — DDPM, Spaced DDPM, DDIM

3.1 Forward vs generation, and the DDPM chain

3.2 Collapsing Gaussian steps → Spaced DDPM

3.3 DDIM: a deterministic trajectory between arbitrary noise levels

3.4 Spaced DDPM vs DDIM, and the modern picture

Chapter 4: The Diffusion Handbook (Parts I–X)

Chapter 5: Image conditioning — vision encoder + image projector

5.1 Two kinds of conditioning: semantic vs pixel-aligned

5.2 Image conditioning as tokens: $I = P_{\text{img}}(E_{\text{vision}}(x_0))$

5.3 Why encode the *clean* image, and why a projector at all

5.4 Where I actually enters: decoupled cross-attention

5.5 Tokens vs channel-concat: choosing by whether the pixels line up

Diffusion Learnings

A running, study-from-cold notebook. One session → one chapter. Every equation in LaTeX, every idea built up from first principles with worked examples.

Table of contents

Chapter 1 — Image codecs: VAE → VQ-VAE → VQGAN → MoVQ (and where latent diffusion sits)

- 1.1 The big picture: two separate problems
- 1.2 The evolution of the image codec
- 1.3 Architecturally, these are mostly CNN/ResNet autoencoders
- 1.4 What makes a VAE a VAE?
- 1.5 The VAE problem: KL can become too successful (posterior collapse)
- 1.6 VQ-VAE's radically different idea: make z discrete
- 1.7 The VQ-VAE training problem: nearest neighbour kills the gradient
- 1.8 Straight-through estimator: lie in backward, not forward
- 1.9 VQ-VAE has three learning jobs
- 1.10 Why codebook + commitment separately?
- 1.11 EMA gives an even cleaner interpretation of codebook learning
- 1.12 Why pixel MSE alone can produce blurry reconstruction
- 1.13 Three complementary reconstruction pressures
- 1.14 LPIPS from beginning to end
- 1.15 PatchGAN is a CNN critic of local realism
- 1.16 VQGAN = VQ representation + perceptual realism
- 1.17 MoVQ: what was left to improve?
- 1.18 MoVQ also injects the codes repeatedly into the decoder
- 1.19 The important latent distinction (three different "latents")
- 1.20 This explains the evolution of the actual generation systems
- 1.21 Why the DALL·E 2 prior can help diversity
- 1.22 The final mental map

Chapter 2 — Training mechanics: normalization, CNNs, and the Adam/AdamW family

- 2.1 CNN translation behavior: equivariance vs invariance
- 2.2 Normalization: BatchNorm / GroupNorm / LayerNorm / RMSNorm
- 2.3 Adam: an adaptive per-coordinate learning rate
- 2.4 Why large normalized weights can still be undesirable
- 2.5 L2 regularization

- 2.6 Why AdamW exists
- 2.7 Weight decay does not normally flip signs

Chapter 3 — Diffusion sampling: DDPM, Spaced DDPM, DDIM

- 3.1 Forward vs generation, and the DDPM chain
- 3.2 Collapsing Gaussian steps → Spaced DDPM
- 3.3 DDIM: a deterministic trajectory between arbitrary noise levels
- 3.4 Spaced DDPM vs DDIM, and the modern picture

Chapter 4 — The Diffusion Handbook (Parts I–X) *(pending source recovery — see note)*

Chapter 5 — Image conditioning: vision encoder + image projector

- 5.1 Two kinds of conditioning: semantic vs pixel-aligned
- 5.2 Image conditioning as tokens: $I = P_{\text{img}}(E_{\text{vision}}(x_0))$
- 5.3 Why encode the *clean* image, and why a projector at all
- 5.4 Where I actually enters: decoupled cross-attention
- 5.5 Tokens vs channel-concat: choosing by whether the pixels line up

---

Chapter 1: Image codecs — VAE → VQ-VAE → VQGAN → MoVQ (and where latent diffusion sits)

This chapter builds the clean mental model tying together the whole image-codec story — from VAE to VQ-VAE to VQGAN to MoVQ — plus the losses, straight-through gradients, LPIPS/GAN, and *why* these systems evolved the way they did. The recurring theme: modern image generators separate **compressing images** from **generating good compressed representations**, and almost every design choice below is an answer to "how should we compress an image so reconstruction is excellent, the latent is easy to model, and generation is cheap?"

1.1 The big picture: two separate problems

Modern image generation often separates:

$$\boxed{\text{compress/decompress images}}$$

from:

$$\boxed{\text{generate good compressed representations}}.$$

The first is the image codec:

$$x \xrightarrow{E} z \xrightarrow{D} \hat{x}.$$

The second can be diffusion, flow matching, autoregression, etc.:

$$\epsilon \xrightarrow[\text{text}]{\text{generative model}} z \xrightarrow{D} x.$$

This distinction explains Stable Diffusion:

$$\text{text} + \text{noise} \xrightarrow{\text{latent diffusion}} z \xrightarrow{\text{VAE decoder}} x.$$

And Kandinsky 2.1:

$$\text{text} \to z_I^{CLIP} \to \text{diffusion in MoVQ latent space} \to z^{MoVQ} \to \text{MoVQ}$$

MoVQ isn't doing the text-to-image reasoning. It's primarily an image compressor/decompressor.

1.2 The evolution of the image codec

The easiest history to remember is:

$$\boxed{\text{VAE} \to \text{VQ-VAE} \to \text{VQGAN} \to \text{MoVQ}}$$

but these aren't simply newer versions replacing one another. They make different bottleneck choices.

VAE

$$x \xrightarrow{\text{CNN encoder}} (\mu, \sigma)$$

sample:

$$z = \mu + \sigma\epsilon, \qquad \epsilon \sim N(0, I)$$

then:

$$z \xrightarrow{\text{CNN decoder}} \hat{x}.$$

Continuous stochastic latent.

VQ-VAE

$$x \xrightarrow{\text{CNN encoder}} z_e \xrightarrow{\text{nearest codebook vector}} z_q \xrightarrow{\text{CNN decoder}} \hat{x}.$$

Discrete latent.

VQGAN

Basically:

$$\boxed{\text{stronger VQ-VAE} + \text{perceptual reconstruction} + \text{adversarial trainin}}$$

to get much better perceptual image quality.

MoVQ

Builds on the VQGAN family but improves how much information discrete codes can represent and how the decoder uses them.

1.3 Architecturally, these are mostly CNN/ResNet autoencoders

For images, the classic structure is approximately:

$$\begin{gathered} 256 \times 256 \times 3 \\ \downarrow \quad \text{Conv} + \text{ResBlocks} \\ 128 \times 128 \times C \\ \downarrow \end{gathered}$$

$$64 \times 64 \times 2C$$

$$\downarrow$$

$$32 \times 32 \times 4C$$

$$\downarrow$$

$$\boxed{z}.$$

Then the decoder mirrors this:

$$z \to 32^2 \to 64^2 \to 128^2 \to 256^2 \times 3.$$

A typical residual block looks approximately like:

$$F \to GN \to SiLU/ReLU \to Conv_{3\times 3} \to GN \to activation \to Conv$$

plus:

$$+ F$$

for the residual connection.

Different models use somewhat different exact blocks, but that's the overall architecture. Transformers can and increasingly do appear in image tokenizers/codecs, but CNNs remain attractive because the codec mostly needs efficient local processing. Global self-attention over every high-resolution spatial position is expensive.

Hence the very natural division:

$$\boxed{\text{efficient CNN/hybrid codec} + \text{powerful Transformer/DiT in compres}}$$

1.4 What makes a VAE a VAE?

The encoder doesn't produce one deterministic z. It produces a distribution:

$$q_\phi(z|x) = N(\mu(x), \mathrm{diag}(\sigma^2(x))).$$

Then:

$$z = \mu + \sigma\epsilon.$$

There are two competing objectives. Reconstruct:

$$L_{\text{rec}}(x, \hat{x})$$

while making latent distributions resemble:

$$p(z) = N(0, I).$$

So:

$$\boxed{L_{\text{VAE}} = L_{\text{rec}} + \beta\, D_{KL}(q_\phi(z|x)\|N(0,I)).}$$

The KL term makes latent space organized and sampleable. Instead of every image being mapped somewhere arbitrary:

```
cat      -> [500, -82, 1300, ...]
dog      -> [-9000, 72, 4, ...]
chair    -> [18273, -3, ...]
```

we encourage the aggregate representation toward a known, manageable region. This is particularly useful when another generative model must operate in that latent space.

1.5 The VAE problem: KL can become too successful (posterior collapse)

Suppose the decoder is powerful enough to model x without using z. Then compare:

$$\text{use } z: \quad L_{\text{rec}} = 1.0,\ KL = 4$$

versus:

$$\text{ignore } z: \quad L_{\text{rec}} = 1.2,\ KL = 0.$$

The second has lower total loss. Therefore:

$$q(z|x) \to p(z).$$

But then $q(z|x)$ barely depends on x, so z contains little information. That's:

$$\boxed{\text{posterior collapse}}.$$

This is especially problematic with very powerful autoregressive decoders.

Compact recap (the tug-of-war). Posterior collapse in VAEs: KL pushes $q(z|x)$ toward the prior, which can erase information about x. Reconstruction losses push the opposite way by requiring z to preserve enough information to rebuild the input. LPIPS helps preserve perceptually meaningful structure; GAN loss pushes reconstructions toward looking like real images.

1.6 VQ-VAE's radically different idea: make z discrete

Create a dictionary:

$$\boxed{\mathcal{E} = \{e_1, e_2, \ldots, e_K\}}$$

where $e_k \in \mathbb{R}^D$. The encoder outputs:

$$z_e \in \mathbb{R}^{h \times w \times D}.$$

At every spatial position $z_e[i,j] \in \mathbb{R}^D$, find:

$$\boxed{k^* = \arg\min_k \|z_e[i,j] - e_k\|^2.}$$

Then replace $z_e[i,j]$ with:

$$\boxed{z_q[i,j] = e_{k^*}.}$$

So the decoder never sees the arbitrary continuous encoder vector. It sees a member of a finite learned vocabulary. You can think of the codes as learned visual words, although they need not correspond cleanly to human concepts.

1.7 The VQ-VAE training problem: nearest neighbour kills the gradient

The quantizer is:

$$Q(z_e) = e_{\arg\min_k \|z_e - e_k\|}.$$

It's piecewise constant. If $e_1 = 0,\ e_2 = 10$, then approximately:

$$Q(z) = \begin{cases} 0 & z < 5 \\ 10 & z > 5. \end{cases}$$

Almost everywhere:

$$\boxed{\frac{\partial Q}{\partial z} = 0.}$$

Therefore the true reconstruction gradient would give:

$$\frac{\partial L}{\partial z_e} = \frac{\partial L}{\partial z_q} \underbrace{\frac{\partial z_q}{\partial z_e}}_{0} = 0.$$

The encoder couldn't learn from reconstruction.

1.8 Straight-through estimator: lie in backward, not forward

Forward remains genuinely discrete:

$$\boxed{z_q = Q(z_e)}.$$

But backward pretends:

$$\boxed{\frac{\partial z_q}{\partial z_e} \approx I.}$$

Therefore:

$$\boxed{\frac{\partial L}{\partial z_e} \approx \frac{\partial L}{\partial z_q}.}$$

This is not the mathematically correct local derivative — it's a useful surrogate. The decoder effectively says: *if my latent input moved this direction, reconstruction would improve.* STE copies that signal to the encoder. Then z_e moves continuously until potentially crossing a nearest-neighbour boundary:

$$e_{17} \to e_{42}.$$

So the surrogate gradient eventually changes the actual discrete decision. That's why it works despite being biased.

1.9 VQ-VAE has three learning jobs

This is the cleanest way to remember its losses. We have three components:

$$\boxed{\text{encoder}} \qquad \boxed{\text{codebook}} \qquad \boxed{\text{decoder}}.$$

Job 1: reconstruct the image

$$L_{\text{rec}} = \|x - D(z_q)\|^2.$$

This trains the decoder directly and the encoder through STE.

Job 2: make dictionary vectors follow encoder clusters

$$\boxed{L_{\text{code}} = \|\operatorname{sg}[z_e] - e_k\|^2.}$$

Only e_k gets this gradient.

Job 3: make encoder commit to the dictionary

$$\boxed{L_{\text{commit}} = \beta\|z_e - \operatorname{sg}[e_k]\|^2.}$$

Only the encoder gets this gradient. Together:

$$\boxed{L_{\text{VQ}} = L_{\text{rec}} + L_{\text{code}} + \beta L_{\text{commit}}.}$$

1.10 Why codebook + commitment separately?

Because they represent opposite responsibilities:

$$\boxed{e_k \to z_e}$$

means *dictionary: move toward the encoder representations assigned to you.* While:

$$\boxed{z_e \to e_k}$$

means *encoder: don't move arbitrarily far from the discrete representation your decoder actually receives.* The second can be weaker ($\beta < 1$). This means: give the encoder freedom to discover useful representations, let the dictionary chase those representations strongly, but keep enough commitment that encoder outputs and actual quantized inputs don't diverge.

If β is huge: the encoder gets trapped around existing codes. If β is too small: the encoder can outrun/drift away from its codebook. That's why separating the two is useful.

1.11 EMA gives an even cleaner interpretation of codebook learning

Instead of gradient-descent updating code e_k, collect encoder outputs assigned to it:

$$z_e^{(1)}, z_e^{(2)}, \dots$$

Calculate their mean $\bar{z}_k$. Then approximately:

$$\boxed{e_k \leftarrow \gamma e_k + (1-\gamma)\bar{z}_k.}$$

So the codebook behaves like online k-means. Then responsibilities become very clean:

$$\boxed{\text{encoder} \xleftarrow{\text{gradient}} L_{\text{rec}} + L_{\text{commit}}}$$

$$\boxed{\text{decoder} \xleftarrow{\text{gradient}} L_{\text{rec}}}$$

$$\boxed{\text{codebook} \xleftarrow{\text{EMA}} \text{encoder cluster centers.}}$$

1.12 Why pixel MSE alone can produce blurry reconstruction

Compression means $x \to z$ throws away information. Suppose after compression the decoder can't know whether the original contained texture A, B, or C. For MSE:

$$\hat{x} = \arg\min_y E[\|X - y\|^2 \mid z].$$

The optimum is:

$$\boxed{\hat{x} = E[X \mid z].}$$

That's the conditional mean. If hair could occur at several slightly different locations, averaging them can produce:

$$\boxed{\text{smooth/blurry hair}}.$$

This isn't because MSE is a bad metric mathematically. It's because under uncertainty, squared error rewards averaging. With unlimited information capacity and perfect reconstruction, this issue could largely disappear. Compression makes it important.

Compact recap. Why MSE blurs: if the latent loses information and one z is compatible with multiple possible images, minimizing squared error gives the conditional mean $E[x|z]$. Averaging multiple plausible sharp images can look blurry. LPIPS changes the geometry by comparing deep visual features rather than exact pixels, so a coherent sharp structure can sometimes be preferable to a pixelwise average. GAN strengthens the pressure toward realistic-looking outputs.

1.13 Three complementary reconstruction pressures

This became a major idea in VQGAN/perceptual autoencoders.

Pixel loss

$$L_{\text{pixel}} = \|x - \hat{x}\|_1 \quad \text{or} \quad \|x - \hat{x}\|_2^2.$$

Asks: "*Are these exact pixel values close?*" — an excellent anchor to the particular target.

LPIPS

$$L_{\text{LPIPS}} = d_{\text{perceptual}}(x, \hat{x}).$$

Asks: "*Does this perceptually resemble this target?*" — allows some pixel disagreement while preserving perceptually important structure.

GAN

$$L_{\text{GAN}}$$

asks: "*Does this look like it came from the real-image distribution?*" — it does not ask directly whether every hair matches the original hair. So:

$$\boxed{\text{pixel} = \text{exact target anchor}}$$

$$\boxed{\text{LPIPS} = \text{perceptual target similarity}}$$

$$\boxed{\text{GAN} = \text{realism/manifold pressure.}}$$

This is why LPIPS and GAN are not redundant.

1.14 LPIPS from beginning to end

Take $x, \hat{x}$. Run both through the same frozen pretrained visual network:

$$x \to \phi_1(x), \ldots, \phi_L(x)$$

$$\hat{x} \to \phi_1(\hat{x}), \ldots, \phi_L(\hat{x}).$$

At layer l: $\phi_l(x) \in \mathbb{R}^{H_l \times W_l \times C_l}$. Normalize feature vectors along channels:

$$\tilde{\phi}_l(x)_{hw} = \frac{\phi_l(x)_{hw}}{\|\phi_l(x)_{hw}\|}.$$

Calculate:

$$\Delta_{lhwc} = \tilde{\phi}_l(x)_{hwc} - \tilde{\phi}_l(\hat{x})_{hwc}.$$

Square and weight channels:

$$d_{lhw} = \sum_c w_{lc}^2 \Delta_{lhwc}^2.$$

Average spatially:

$$D_l = \frac{1}{H_l W_l} \sum_{h,w} d_{lhw}.$$

Sum layers:

$$\boxed{LPIPS(x, \hat{x}) = \sum_l D_l.}$$

The w_l's were calibrated using human perceptual judgments so that the resulting distance better predicts which image humans perceive as closer to a reference. During VQGAN/VAE training, LPIPS is frozen. But:

$$\boxed{\text{frozen} \neq \text{no gradient.}}$$

Gradient flows:

$$L_{\text{LPIPS}} \to \hat{x} \to \text{decoder} \to \cdots$$

without updating LPIPS's own weights.

Compact recap. LPIPS: pass both images through a frozen CNN, take feature vectors at several layers and spatial locations, normalize those vectors, compare their differences, apply learned per-channel perceptual weights, average over space, and sum across layers. Because normalized squared Euclidean distance is closely related to cosine distance, LPIPS is effectively comparing feature direction with learned importance across channels.

CNN translation behavior. Convolutional feature maps are mainly translation-equivariant: shift the image, and the feature map shifts. Invariance appears when location is deliberately discarded, such as through global pooling. LPIPS keeps spatial feature maps, so it is not fully translation invariant.

1.15 PatchGAN is a CNN critic of local realism

Input: $256 \times 256 \times 3$. The CNN discriminator produces progressively richer features:

$$256^2 \times 3 \to 128^2 \times 64 \to 64^2 \times 128 \to 32^2 \times 256 \to \cdots$$

At one spatial location there may be $F[i, j] \in \mathbb{R}^{512}$. That's not just one number — it is a rich description of a large receptive field in the original image. A final convolution turns $\mathbb{R}^{512} \to \mathbb{R}$ at every location. So the final discriminator output is something like:

$$D(x) \in \mathbb{R}^{30 \times 30}.$$

Every element asks approximately:

> "Does the corresponding receptive field look locally real?"

The patches overlap — you don't manually cut the image — and convolutional weight sharing means the same learned realism detectors operate across the image.

1.16 VQGAN = VQ representation + perceptual realism

This is the conceptual jump.

VQ-VAE

$$x \to E \to z_e \to VQ \to z_q \to D \to \hat{x}$$

primarily reconstruction + VQ losses.

VQGAN — same broad architecture:

$$x \to E \to z_e \to VQ \to z_q \to D \to \hat{x}$$

but train reconstruction using something closer to:

$$\boxed{L = L_{\text{pixel/perceptual}} + L_{\text{codebook}} + L_{\text{commitment}} + \lambda L_{\text{GAN}}.}$$

Now the codec is allowed to use its limited bits for perceptually important information rather than obsessing entirely over exact pixel errors.

1.17 MoVQ: what was left to improve?

A VQGAN might map nearby/similar encoder vectors to the same code:

$$z_e^A \to e_{17} \qquad z_e^B \to e_{17}.$$

Then the decoder sees exactly e_{17} in both places. This can limit local representational richness and contribute to repeated patterns/artifacts. MoVQ addresses this in two notable ways.

More combinations per spatial position. Instead of one 256-D vector selecting one code, split:

$$256 = 64 + 64 + 64 + 64.$$

Then:

$$z_e[i, j] = [z^1, z^2, z^3, z^4].$$

Each selects independently (k_1, k_2, k_3, k_4). If $K = 1024$, you have:

$$\boxed{K^4}$$

possible combinations at each spatial location without storing a literal K^4-entry codebook.

1.18 MoVQ also injects the codes repeatedly into the decoder

Instead of only $z_q \to D$, it uses:

$$\boxed{F_l = \gamma_l(z_q) \odot GN(F_{l-1}) + \beta_l(z_q).}$$

Here F_{l-1} is the decoder's current feature tensor. GroupNorm normalizes it. Then learned 1×1 convolutions produce spatially varying $\gamma_l(z_q)$ and $\beta_l(z_q)$: γ scales features, β shifts features. Therefore the discrete representation repeatedly tells the decoder: "*at this spatial region, amplify/suppress/shift these channels this way.*" It's much stronger conditioning than simply giving z_q once at the bottom.

1.19 The important latent distinction (three different "latents")

This was one of the biggest conceptual points from the entire discussion. There are at least three completely different things people casually call "latent."

CLIP image latent z_I^{CLIP} — global semantic representation:

$$\boxed{\text{"What kind of image/content is this?"}}$$

Not designed to reconstruct every pixel.

VAE / VQ / MoVQ latent $z^{VAE},\ z^{VQ},\ z^{MoVQ}$ — spatial compressed image representation:

$$\boxed{\text{"Compressed information from which I can reconstruct the image."}}$$

Diffusion latent state z_t — a noisy intermediate state:

$$z_t = \sqrt{\bar{\alpha}_t}\, z_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon.$$

Totally different from the other two.

1.20 This explains the evolution of the actual generation systems

DALL·E 1 — no diffusion:

$$\text{text} \rightarrow \text{autoregressive Transformer} \rightarrow \text{discrete dVAE image tokens} -$$

GLIDE — no image autoencoder bottleneck for the main generation:

$$\text{text} \rightarrow \text{pixel diffusion} \rightarrow 64^2 \rightarrow \text{diffusion SR}.$$

DALL·E 2 / unCLIP:

$$\text{text} \xrightarrow{CLIP} z_T \xrightarrow{\text{diffusion prior}} z_I^{CLIP}$$

then:

$$z_I^{CLIP} \rightarrow \text{pixel diffusion at } 64^2 \rightarrow \text{diffusion SR} \rightarrow 1024^2.$$

So DALL·E 2's main image diffusion happens in pixels, not a VAE spatial latent like Stable Diffusion.

Karlo — similar unCLIP concept:

$$\text{text} \rightarrow CLIP\ image\ prior \rightarrow 64^2\ pixel\ diffusion \rightarrow fast\ SR.$$

Stable Diffusion — different:

$$x \to \boxed{\text{VAE latent}}$$

and diffusion learns $\text{t} + \text{noise} \to z^{VAE}$, then:

$$z^{VAE} \xrightarrow{\text{VAE decoder}} x.$$

The expensive iterative process happens in compressed space.

Kandinsky 2.1 — combines ideas:

$$\text{text} \to z_I^{CLIP}$$

then:

$$z_I^{CLIP} + \text{text} \to \text{diffusion in MoVQ spatial latent}$$

then:

$$z^{MoVQ} \xrightarrow{\text{MoVQ decoder}} x.$$

So conceptually:

$$\boxed{\text{unCLIP semantic prior} + \text{Stable-Diffusion-style latent generation} +}$$

1.21 Why the DALL·E 2 prior can help diversity

A direct model learns $p(x|c)$. DALL·E 2 explicitly factorizes:

$$\boxed{p(x|c) = \int p(x|z_I, c)\, p(z_I|c)\, dz_I.}$$

So for "a corgi on a beach" the prior can first sample:

$$z_I^1 = \text{close-up} \qquad z_I^2 = \text{running through water} \qquad z_I^3 = \text{beside a surfb}$$

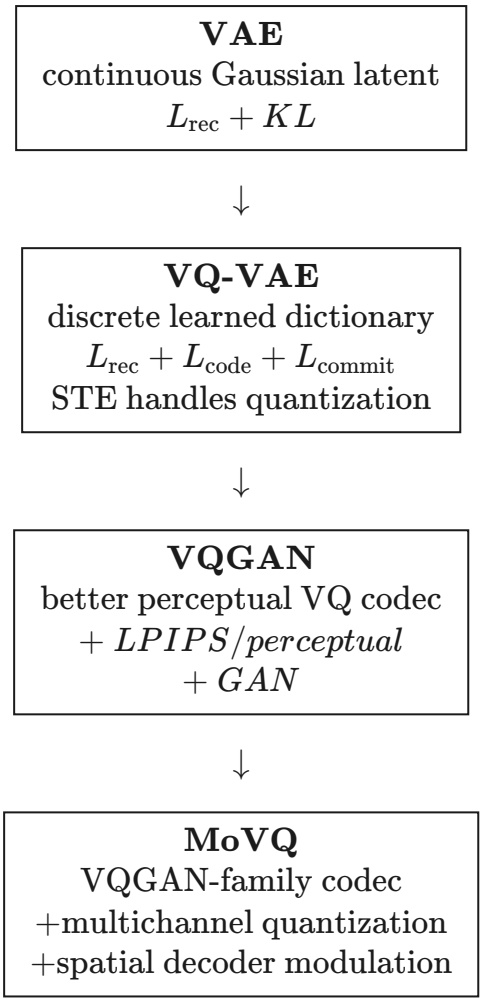
Then the decoder renders the selected interpretation. Conceptually:

$$\boxed{\text{prior chooses visual interpretation}}$$

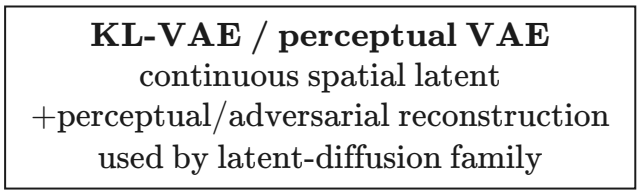
then the decoder realizes it. A sufficiently powerful direct model can learn this implicitly, so the prior is not mathematically necessary. It is an architectural factorization / inductive bias.

1.22 The final mental map

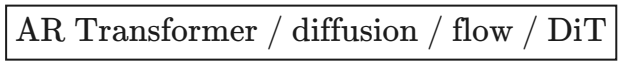
If you remember only this, remember:



while the alternative branch is:

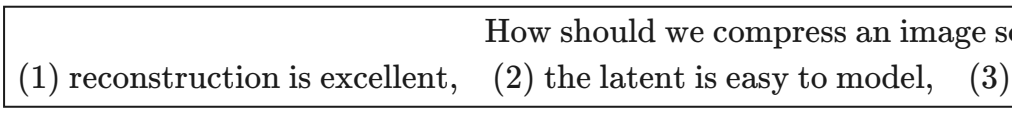


And on top of whichever codec you choose, you can train a powerful generative model:



to learn the distribution of those latents.

The conceptual evolution is therefore not simply "VAE got replaced by VQ-VAE got replaced by VQGAN got replaced by MoVQ." It is:



VAE and VQ-style codecs give different answers to that tradeoff, and modern image generators build the expensive semantic/generative model on top of that compressed representation rather than repeatedly reasoning over full-resolution RGB pixels.

> **Key takeaway.** Separate the two problems: a codec ($x \to z \to \hat{x}$) and a generator over z. VAE regularizes a continuous latent with KL (risking posterior collapse); VQ-VAE swaps to a discrete codebook, needing the straight-through estimator plus codebook + commitment losses; VQGAN adds LPIPS + PatchGAN so limited bits buy *perceptual* quality; MoVQ enriches codes (multi-channel quantization $\to K^4$ combinations) and modulates the decoder with them ($F_l = \gamma_l(z_q) \odot GN(F_{l-1}) + \beta_l(z_q)$). Never conflate the three "latents" — CLIP semantic z_I^{CLIP}, spatial codec $z^{VAE/VQ}$, and noisy diffusion state z_t.

Chapter 2: Training mechanics — normalization, CNNs, and the Adam/AdamW family

This chapter collects the optimizer and normalization machinery that sits under almost every model in this notebook. The through-line: normalization decides *what statistic* a layer is invariant to, and the optimizer decides *how* a parameter moves — and the two interact (normalized nets care about weight *direction*, which is why weight decay is handled the way it is).

2.1 CNN translation behavior: equivariance vs invariance

Convolutional feature maps are mainly translation-equivariant: shift the image, and the feature map shifts. Invariance appears when location is deliberately discarded, such as through global pooling. LPIPS keeps spatial feature maps, so it is not fully translation invariant.

2.2 Normalization: BatchNorm / GroupNorm / LayerNorm / RMSNorm

BatchNorm normalizes each channel using statistics across batch and spatial positions. GroupNorm avoids dependence on the batch by normalizing groups of channels within each image across channel-group, height, and width. LayerNorm normalizes each token across its hidden dimensions. RMSNorm mainly normalizes vector magnitude and skips explicit mean subtraction.

2.3 Adam: an adaptive per-coordinate learning rate

For each parameter, Adam tracks

$$m_t \approx \text{smoothed gradient}$$

and

$$v_t \approx \text{smoothed squared gradient}.$$

The update is roughly

$$-\eta \frac{m_t}{\sqrt{v_t}}.$$

So Adam behaves like an adaptive learning rate per parameter coordinate. It normalizes gradient scale; it does not directly control the size of the parameter itself.

2.4 Why large normalized weights can still be undesirable

If normalization makes w and $100w$ behave almost identically in the forward pass, the norm of w can drift. But for a fixed-size update Δw, a larger $\|w\|$ means a smaller change in its direction:

$$\frac{\|\Delta w\|}{\|w\|} \downarrow .$$

In normalized networks, direction can matter more than magnitude, so this can effectively slow learning.

2.5 L2 regularization

Adding

$$\frac{\lambda}{2}\|w\|^2$$

to the loss gives gradient

$$\lambda w,$$

because

$$\nabla_w \frac{\lambda}{2}\|w\|^2 = \lambda w.$$

With plain SGD this creates

$$w_{t+1} = w_t - \eta g - \eta\lambda w_t.$$

2.6 Why AdamW exists

If you put λw inside Adam's gradient, Adam also rescales the regularization according to past gradient statistics m, v. Then equal-sized weights can shrink by different amounts. AdamW separates them:

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta\lambda w_t.$$

So gradient adaptation depends on gradient history, while weight shrinkage depends directly on w.

2.7 Weight decay does not normally flip signs

Ignoring the task gradient,

$$w_{t+1} = (1 - \eta\lambda) w_t.$$

For the usual case $0 < \eta\lambda \ll 1$, this gently shrinks w toward zero without crossing it.

> **Key takeaway.** Adam adapts the *step per coordinate* from gradient history ($m_t/\sqrt{v_t}$); weight decay controls *weight magnitude* directly. AdamW keeps those two jobs separate ($-\eta\lambda w_t$ applied outside the adaptive term) so that in a normalized network — where direction matters more than magnitude — shrinkage is uniform and predictable rather than rescaled by gradient statistics.

Chapter 3: Diffusion sampling — DDPM, Spaced DDPM, DDIM

This chapter distinguishes two families of "skip-ahead" samplers that look identical on a trajectory plot but arise from completely different reasoning: a *shorter stochastic Markov chain* (Spaced DDPM) versus a *deterministic change-of-variables* (DDIM).

3.1 Forward vs generation, and the DDPM chain

For diffusion, the key distinction is:

$$0 \to T$$

is the forward noising process, and

$$T \to 0$$

is generation. A standard DDPM has

$$x_0 \to x_1 \to \cdots \to x_T$$

with

$$\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i.$$

3.2 Collapsing Gaussian steps → Spaced DDPM

Many Gaussian forward steps can be collapsed because Gaussians compose. For example,

$$\frac{\alpha_{400}}{\bar{\alpha}_{200}} = \alpha_{201} \cdots \alpha_{400}$$

is exactly the total signal retention from timestep 200 to 400. That lets Spaced DDPM define a new shorter Markov chain using

$$\alpha'_k = \frac{\bar{\alpha}_{t_k}}{\bar{\alpha}_{t_{k-1}}}, \qquad \beta'_k = 1 - \alpha'_k.$$

Then reverse sampling can genuinely jump

$$x_{1000} \to x_{800} \to x_{600} \to \cdots$$

using the Gaussian posterior of that new shortened DDPM and fresh stochastic noise.

3.3 DDIM: a deterministic trajectory between arbitrary noise levels

DDIM gets the same-looking jumps for a different reason. At x_t, predict $\hat{\epsilon}$, infer

$$\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\hat{\epsilon}}{\sqrt{\bar{\alpha}_t}},$$

and then directly place the sample at an earlier noise level s:

$$x_s = \sqrt{\bar{\alpha}_s}\hat{x}_0 + \sqrt{1-\bar{\alpha}_s}\hat{\epsilon}$$

for deterministic DDIM.

3.4 Spaced DDPM vs DDIM, and the modern picture

So both can internally use $\hat{x}_0$, but:

$$\boxed{\text{Spaced DDPM} = \text{shorter stochastic Markov chain}}$$

whereas

$$\boxed{\text{DDIM} = \text{deterministic/controlled trajectory between arbitrary noise}}$$

And the modern broader picture is that the field has increasingly moved toward latent autoencoders + Transformer backbones + diffusion/flow-matching or rectified-flow objectives, with old-style Spaced DDPM being more historically important than central to the newest systems.

> **Key takeaway.** Same jumps, different guarantees. Spaced DDPM builds a genuine shorter Markov chain ($\alpha'_k = \bar{\alpha}_{t_k}/\bar{\alpha}_{t_{k-1}}$) and still injects fresh noise each step. DDIM fixes an $\hat{x}_0$ estimate and re-places it at any noise level deterministically — one roll of the dice at x_T, then a fixed recipe.

Chapter 4: The Diffusion Handbook (Parts I–X)

> ⚠️ **Source not yet recovered — content pending.**
>
> `diffusion-handbook.pdf` is the project's 84-page reference (Parts I–X: the probabilistic core, training/sampling/distillation recipes, inference optimisation, real-time video, world models, evaluation, and a corrections log). It exists **only as a PDF** — a prior session searched the whole project and the home directory and found **no markdown/HTML/TeX source**. It cannot be copied in "word for word" until we either (a) install a PDF text tool (`poppler`) and extract/transcribe it, or (b) locate/re-supply its original source. Until then this chapter is a placeholder so the rest of the notebook is complete and the handbook is not silently dropped. **The PDF itself is untouched and safe.**

Chapter 5: Image conditioning — vision encoder + image projector

Text conditioning answers "what should the model *make*?" with a caption. But a huge amount of practice is conditioning on **another image**: "make it in *this* style," "keep *this* person's face," "use *this* as a reference." This chapter builds the single most common recipe for that — turning a reference image into a handful of conditioning tokens the denoiser can attend to — and shows why it is written the way the seed formula is written:

$$I = P_{\text{img}}\big(E_{\text{vision}}(x_0)\big).$$

5.1 Two kinds of conditioning: semantic vs pixel-aligned

Before any formula, one distinction decides *everything* about how you wire an image in. Ask: **do the conditioning pixels line up, cell-for-cell, with the pixels I am generating?**

- **Pixel-aligned** — inpainting mask, a depth/edge map, a low-res image for super-resolution. Position (i, j) of the condition means the same place as position (i, j) of the output. The tensor's own indexing carries the correspondence, so the right move is to **concatenate on the channel axis** and let the first convolution read both at once. No learning of "where things go" is needed.
- **Semantic / non-aligned** — a style reference, an identity photo, "the vibe of this painting." There is *no* pixel correspondence: the reference face may be centered while the output face is off to the left. Concatenating by position would be meaningless. Instead you compress the reference into a small set of **content tokens** and let **cross-attention** decide, per output location, what to borrow.

The seed formula lives entirely in the **second** world. I is a bag of semantic tokens, not a spatial map.

5.2 Image conditioning as tokens: $I = P_{\text{img}}(E_{\text{vision}}(x_0))$

Read the formula right-to-left — it is a two-stage pipeline:

$$x_0 \xrightarrow{E_{\text{vision}}} f \xrightarrow{P_{\text{img}}} I.$$

- x_0 — the **clean reference image** (not a noisy latent; see §5.3).
- E_{vision} — a **frozen, pretrained vision encoder**, typically a CLIP or DINO image tower. It maps the image to features f. Concretely a CLIP ViT-L/14 gives you either a single pooled embedding ($f \in \mathbb{R}^{1024}$) or the full patch sequence (e.g. $f \in \mathbb{R}^{257\times1024}$ — a `[CLS]` token plus 16×16 patch

tokens). It is frozen because it already knows "what images mean"; retraining it would waste that and risk collapse.

- P_{img} — a **small learned image projector**. Its whole job is to bridge the *domain gap*: E_{vision} lives in CLIP/DINO space, but the denoiser's cross-attention was built to consume **text-encoder** tokens of some width d. P_{img} maps vision features into that same d and, usually, into a fixed small **number** of tokens.
- I — the resulting **image conditioning tokens**, shape $(B,\ M,\ d)$ with M small (IP-Adapter's resampler famously produces $M = 16$ or 64). These are the key/value sequence the denoiser cross-attends to.

P_{img} is not always a bare linear layer. When E_{vision} emits a long patch sequence, P_{img} is often a small **resampler / Perceiver-style attention block** with M learned query tokens:

```
# P_img as a learnable resampler (schematic)
q = learned_queries            # (M, d)      M ≪ 257
f = E_vision(x0)               # (257, 1024) frozen features
I = cross_attn(q, kv=proj(f))  # (M, d)      queries pull a fixed-size
summary
```

So P_{img} does two things at once: **project** (width $1024 \to d$) and **summarize** ($257 \to M$ tokens).

5.3 Why encode the *clean* image, and why a projector at all

Two design choices in the formula are easy to skip past.

Why x_0 and not a noisy x_t? The reference is a *fixed side input*, not part of the trajectory being denoised. We want its full semantic content, so we feed the clean image and encode it **once**, outside the sampling loop. Its tokens I are then reused unchanged at every timestep — cheap, and stable.

Why P_{img} at all — why not feed $E_{\text{vision}}(x_0)$ straight in? Two mismatches make that impossible:

1. **Width.** CLIP features are (say) 1024-d; the cross-attention projections W_K, W_V expect width d (the text-token width). A linear map is the minimum fix.
2. **Semantics/scale.** CLIP-space directions are not text-encoder-space directions, and their scale is different. A tiny trained projector aligns the two distributions so the frozen attention weights actually "understand" the injected tokens.

That is the whole reason the projector is a *learned* module wrapped around a *frozen* encoder: freeze the expensive part that already works, train only the thin adapter that translates it.

5.4 Where I actually enters: decoupled cross-attention

Naively you could concatenate I onto the text tokens and run one cross-attention. In practice the strong recipe (IP-Adapter) keeps them **separate** — *decoupled cross-attention* — adding a second attention that shares the query but has its **own** W'_K, W'_V for the image tokens:

$$\text{out} = \underbrace{\text{Attn}(Q,\, K_{\text{txt}},\, V_{\text{txt}})}_{\text{original, frozen}} \;+\; \lambda\, \underbrace{\text{Attn}(Q,\, K_{\text{img}},\, V_{\text{img}})}_{\text{new, trained}},$$

with

$$K_{\text{img}} = I\, W'_K, \qquad V_{\text{img}} = I\, W'_V.$$

Why decouple? (i) It lets the base text-to-image model stay **frozen** — only W'_K, W'_V (and P_{img}) are trained, so the adapter is tiny and doesn't degrade the original text behavior. (ii) The scalar λ becomes a clean **image-conditioning strength** knob at inference: $\lambda = 0$ recovers the pure text model, larger λ leans harder on the reference. This is exactly the "one vector or many / add a side stream" pattern generalized to a whole image.

5.5 Tokens vs channel-concat: choosing by whether the pixels line up

Tie §5.1 back to the two mechanisms so the decision is mechanical:

Situation	Correspondence	Mechanism	Why
Inpaint / depth / super-res	pixel-aligned	**channel concat** into `conv_in`	indexing *is* the alignment; no cross-attn needed
Style / identity / reference	none	**tokens via** $I = P_{\text{img}}(E_{\text{vision}}(x_0))$ + cross-attn	attention learns *what* to borrow, not *where*

The failure mode of using the wrong one is instructive: channel-concat a style reference and the model tries to copy it *positionally* (ghosting, mis-registered content); tokenize a depth map and you throw away the precise geometry that only survives in the spatial grid. Match the mechanism to the correspondence.

> **Key takeaway.** $I = P_{\text{img}}(E_{\text{vision}}(x_0))$ is the semantic-image conditioning recipe: a **frozen** vision encoder turns the clean reference into features, a **small trained projector/resampler** maps them to M tokens of the denoiser's width, and those tokens enter through a **decoupled** cross-

attention with its own W'_K, W'_V and a strength scalar λ. Use it only when the reference does **not** align pixel-for-pixel with the output — for aligned conditions, channel-concat and skip the encoder entirely.